

Avaliação Técnica

Desenvolvedor Front End — Angular

Avalia conhecimentos práticos em Angular 17+, RxJS, NgRx/Signals, Angular Material, testes unitários, TypeScript e integração com APIs REST.

 **Prazo de entrega**
4 dias corridos após recebimento

 **Entrega**
Encaminhar este documento com as perguntas respondidas e com o link do código público em sua conta do GitHub

1. TypeScript e Qualidade de Código

1.1. Refatoração

Considerando seus conhecimentos de TypeScript, qualidade de código e boas práticas, quais melhorias você faria no seguinte código:

```
class Produto {
  id: any;
  descricao: any;
  quantidadeEstoque: any;

  constructor(id: any, descricao: any, quantidadeEstoque: any) {
    this.id = id;
    this.descricao = descricao;
    this.quantidadeEstoque = quantidadeEstoque;
  }
}

class Verdeira {
  produtos: any;

  constructor() {
    this.produtos = [
      new Produto(1, 'Maçã', 20),
      new Produto(2, 'Laranja', 0),
      new Produto(3, 'Limão', 20)
    ];
  }

  getDescricaoProduto(produtoId: any) {
    let produto;

    for (let index = 0; index < this.produtos.length; index++) {
      if (this.produtos[index].id == produtoId) {
        produto = this.produtos[index];
      }
    }

    return produto.id + ' - ' + produto.descricao + ' (' + produto.quantidadeEstoque + 'x)';
  }

  hasEstoqueProduto(produtoId: any) {
    let produto;

    for (let index = 0; index < this.produtos.length; index++) {
      if (this.produtos[index].id == produtoId) {
        produto = this.produtos[index];
      }
    }

    if (produto.quantidadeEstoque > 0) {
      return true;
    } else {
      return false;
    }
  }
}
```

RESPOSTA

As principais melhorias aplicadas ao código original são:

- Remoção de **any**, usando tipagem explícita.
- Uso de propriedades **readonly** para dados que não devem ser reatribuídos.
- Uso de **find** no lugar de laços manuais.
- Comparação estrita por tipo.
- Tratamento para produto não encontrado.
- Retorno booleano direto em **hasEstoqueProduto**.
- Uso de template string para melhorar legibilidade.

```
class Produto {
  constructor(
    public readonly id: number,
    public readonly descricao: string,
    public quantidadeEstoque: number
  ) {}
}

class Verdureira {
  readonly produtos: Produto[] = [
    new Produto(1, 'Maçã', 20),
    new Produto(2, 'Laranja', 0),
    new Produto(3, 'Limão', 20),
  ];

  getDescricaoProduto(produtoId: number): string {
    const produto = this.produtos.find((p) => p.id === produtoId);

    return produto
      ? `${produto.id} - ${produto.descricao} (${produto.quantidadeEstoque}x)`
      : 'Produto não encontrado';
  }

  hasEstoqueProduto(produtoId: number): boolean {
    const produto = this.produtos.find((p) => p.id === produtoId);

    return Boolean(produto && produto.quantidadeEstoque > 0);
  }
}
```

1.2. Generics e tipos utilitários

Implemente uma função genérica `filtrarEPaginar<T>` que recebe um array, um predicado de filtro e parâmetros de paginação (página e tamanho). A função deve retornar os itens da página atual e o total de registros filtrados. Use tipagem completa — sem **any**.

Exemplo de parâmetros:

```
filtrarEPaginar<T>(  
  data: T[],  
  filterFn: (item: T) => boolean,  
  params: PaginaParams  
) : Pagina<T>
```

Demonstre o uso com um exemplo concreto (ex: array de usuários com filtro por nome).

RESPOSTA

A função `filtrarEPaginar<T>` recebe uma lista genérica, aplica o filtro informado e retorna apenas os itens da página atual, além do total de registros filtrados.

```
class PaginaParams {  
  constructor(  
    public readonly pagina: number,  
    public readonly tamanho: number  
  ) {}  
}
```

```

class Pagina<T> {
  constructor(
    public readonly itens: T[],
    public readonly totalRegistros: number
  ) {}
}

function filtrarEPaginar<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PaginaParams
): Pagina<T> {
  const dadosFiltrados = data.filter(filterFn);
  const inicio = (params.pagina - 1) * params.tamanho;
  const fim = inicio + params.tamanho;

  return new Pagina(
    dadosFiltrados.slice(inicio, fim),
    dadosFiltrados.length
  );
}

interface Usuario {
  readonly id: number;
  readonly nome: string;
}

const usuarios: Usuario[] = [
  { id: 1, nome: 'Ana' },
  { id: 2, nome: 'Maria' },
  { id: 3, nome: 'Marcos' },
  { id: 4, nome: 'João' },
];

const resultado = filtrarEPaginar(
  usuarios,
  (usuario) => usuario.nome.toLowerCase().includes('mar'),
  new PaginaParams(1, 10)
);

console.log(resultado.itens);
console.log(resultado.totalRegistros);

```

2. Angular — Fundamentos e Reatividade

2.1. Change Detection e OnPush

O componente abaixo usa `ChangeDetectionStrategy.OnPush`, mas o nome não é exibido na tela. Identifique o problema, explique o motivo e proponha a correção — sem alterar a estratégia, sem modificar `PessoaService` e sem remover o `setInterval`.

```

import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy } from '@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
  /** @description Mock de uma busca em API com retorno em 0.5 segundos */
  buscarPorId(id: number) {
    return of({ id, nome: 'João' }).pipe(delay(500));
  }
}

@Component({
  selector: 'app-root',
  providers: [PessoaService],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<h1>{{ texto }}</h1>`,
})

```

```
export class AppComponent implements OnInit, OnDestroy {
  texto: string;
  contador = 0;
  subscriptionBuscarPessoa: Subscription;
  constructor(private readonly pessoaService: PessoaService) {}

  ngOnInit(): void {
    this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
      this.texto = `Nome: ${pessoa.nome}`;
    });

    setInterval(() => this.contador++, 1000);
  }

  ngOnDestroy(): void {
    /** ... */
  }
}
```

RESPOSTA

O problema ocorre porque o componente usa [ChangeDetectionStrategy.OnPush](#) e a propriedade `texto` é alterada dentro de um `subscribe`. Como a atualização acontece de forma assíncrona e a propriedade é um campo comum da classe, o Angular pode não marcar o componente para nova verificação.

Uma correção adequada, sem alterar a estratégia, sem modificar o service e sem remover o `setInterval`, é usar Angular Signals. Ao atualizar o signal com `.set()`, o Angular é notificado da mudança e a tela é atualizada corretamente.

```
import {
  ChangeDetectionStrategy,
  Component,
  Injectable,
  OnDestroy,
  OnInit,
  signal,
} from '@angular/core';
import { delay, of, Subscription } from 'rxjs';

@Injectable()
class PessoaService {
  buscarPorId(id: number) {
    return of({ id, nome: 'João' }).pipe(delay(500));
  }
}

@Component({
  selector: 'app-root',
  providers: [PessoaService],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<h1>{{ texto() }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
  readonly texto = signal('');
  contador = 0;
  private subscriptionBuscarPessoa?: Subscription;

  constructor(private readonly pessoaService: PessoaService) {}

  ngOnInit(): void {
    this.subscriptionBuscarPessoa = this.pessoaService
      .buscarPorId(1)
      .subscribe((pessoa) => {
        this.texto.set(`Nome: ${pessoa.nome}`);
      });

    setInterval(() => this.contador++, 1000);
  }

  ngOnDestroy(): void {
    this.subscriptionBuscarPessoa?.unsubscribe();
  }
}
```

2.2. RxJS — eliminando subscriptions aninhadas

Refatore o código abaixo eliminando o subscribe dentro de subscribe. Use operadores RxJS adequados, evite memory leaks e explique brevemente sua escolha de operador:

```
ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService.buscarPorId(pessoaId).subscribe(pessoa => {
    this.pessoaService.buscarQuantidadeFamiliares(pessoaId).subscribe(qtd => {
      this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
    });
  });
}
```

RESPOSTA

O operador escolhido foi **switchMap**, porque a segunda chamada depende do resultado da primeira. Ele evita **subscribe** dentro de **subscribe**, mantém o fluxo em uma única cadeia RxJS e cancela a requisição anterior caso uma nova execução seja iniciada.

Para evitar memory leaks, a subscription pode ser gerenciada com **takeUntilDestroyed**.

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { map, switchMap } from 'rxjs';

ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService
    .buscarPorId(pessoaId)
    .pipe(
      switchMap((pessoa) =>
        this.pessoaService.buscarQuantidadeFamiliares(pessoaId).pipe(
          map((qtd) => `Nome: ${pessoa.nome} | familiares: ${qtd}`)
        )
      ),
      takeUntilDestroyed(this.destroyRef)
    )
    .subscribe((texto) => {
      this.texto = texto;
    });
}
```

2.3. RxJS — busca com debounce

Implemente um campo de busca reativo em um componente Angular que:

- Aguarde 500 ms após o usuário parar de digitar antes de disparar a requisição (debounce)
- Cancele a requisição anterior caso o usuário digite novamente (evite race condition)
- Exiba um indicador de loading enquanto a requisição está em andamento
- Gerencie a subscription sem memory leak

Mostre o serviço, o componente e o template com async pipe.

RESPOSTA

A implementação usa `debounceTime(500)` para aguardar o usuário parar de digitar antes de buscar, `switchMap` para cancelar a requisição anterior e `takeUntilDestroyed` para evitar vazamento de memória. O template consome os estados com `async pipe`.

SERVICE

```
import { Injectable } from '@angular/core';
import {
  BehaviorSubject,
  catchError,
  delay,
  finalize,
  Observable,
  of,
} from 'rxjs';

export interface Pessoa {
  id: number;
  name: string;
}

@Injectable({ providedIn: 'root' })
export class DebounceSearchService {
  private readonly data: Pessoa[] = [
    { id: 1, name: 'Igor' },
    { id: 2, name: 'Maria' },
    { id: 3, name: 'João' },
    { id: 4, name: 'Ana' },
  ];

  private readonly pessoasSubject = new BehaviorSubject<Pessoa[]>([]);
  private readonly loadingSubject = new BehaviorSubject(false);
  private readonly errorSubject = new BehaviorSubject(false);

  readonly pessoas$ = this.pessoasSubject.asObservable();
  readonly loading$ = this.loadingSubject.asObservable();
  readonly error$ = this.errorSubject.asObservable();

  search(search: string): Observable<Pessoa[]> {
    this.loadingSubject.next(true);
    this.errorSubject.next(false);

    return this.getSmartSearchValues(search).pipe(
      finalize(() => this.loadingSubject.next(false)),
      catchError(() => {
        this.errorSubject.next(true);
        return of([]);
      })
    );
  }

  updateResult(result: Pessoa[]): void {
    this.pessoasSubject.next(result);
  }

  reset(): void {
    this.pessoasSubject.next([]);
    this.errorSubject.next(false);
  }

  private getSmartSearchValues(search: string): Observable<Pessoa[]> {
    const filteredData = this.data.filter((pessoa) =>
      pessoa.name.toLowerCase().includes(search.toLowerCase())
    );

    return of(filteredData).pipe(delay(500));
  }
}
```

COMPONENT

```
import { AsyncPipe } from '@angular/common';
import { Component, inject } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import {
  debounceTime,
  distinctUntilChanged,
  filter,
  switchMap,
  tap,
}
```

```

} from 'rxjs';

@Component({
  selector: 'app-debounce-search',
  standalone: true,
  imports: [ReactiveFormsModule, AsyncPipe],
  templateUrl: './debounce-search.html',
})
export class DebounceSearchComponent {
  private readonly service = inject(DebounceSearchService);

  readonly searchControl = new FormControl('');
  readonly pessoas$ = this.service.pessoas$;
  readonly loading$ = this.service.loading$;
  readonly error$ = this.service.error$;

  constructor() {
    this.searchControl.valueChanges
      .pipe(
        debounceTime(500),
        distinctUntilChanged(),
        tap((value) => {
          if (!value) {
            this.service.reset();
          }
        }),
        filter((value): value is string => Boolean(value)),
        switchMap((value) => this.service.search(value)),
        takeUntilDestroyed()
      )
      .subscribe((result) => {
        this.service.updateResult(result);
      });
  }
}

```

TEMPLATE

```

<input [formControl]="searchControl" placeholder="Buscar..." />

<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Nome</th>
    </tr>
  </thead>

  <tbody>
    @if (loading$ | async) {
      <tr>
        <td colspan="2">Carregando...</td>
      </tr>
    } @else if (error$ | async) {
      <tr>
        <td colspan="2">Erro ao carregar dados</td>
      </tr>
    } @else {
      @for (item of pessoas$ | async; track item.id) {
        <tr>
          <td>{{ item.id }}</td>
          <td>{{ item.name }}</td>
        </tr>
      }
    }
  </tbody>
</table>

```

2.4. Performance — OnPush e trackBy

Considere uma lista com centenas de itens renderizados com `@for` (ngFor). Explique:

- Por que usar `trackBy` melhora a performance e como implementá-lo corretamente
- Como `ChangeDetectionStrategy.OnPush` pode reduzir ciclos desnecessários de detecção neste cenário
- Qual seria o impacto de usar a estratégia `Default` neste caso

RESPOSTA

O **trackBy**, ou **track** no novo control flow com **@for**, melhora a performance porque permite que o Angular identifique cada item por uma chave estável. Assim, quando a lista muda, ele não precisa recriar todos os elementos do DOM; ele atualiza somente os itens realmente alterados.

```
@for (usuario of usuarios; track usuario.id) {  
  <p>{{ usuario.nome }}</p>  
}
```

Com **ChangeDetectionStrategy.OnPush**, o Angular reduz verificações desnecessárias, pois o componente é checado principalmente quando recebe novos inputs, quando ocorre um evento no próprio componente, quando um signal usado no template muda ou quando uma emissão assíncrona observada pelo template acontece.

Com a estratégia **Default**, o Angular executa checagens mais amplas na árvore de componentes sempre que há eventos assíncronos na aplicação. Em telas com muitos componentes ou listas grandes, isso pode gerar mais processamento e piorar a fluidez da interface.

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

Crie um componente de contador de itens no carrinho usando exclusivamente Signals. O componente deve expor:

- Um signal para a lista de itens
- Um computed para o total (quantidade × preço)
- Um método para adicionar e remover itens
- Um output() que emite sempre que o total mudar

RESPOSTA

O componente abaixo usa exclusivamente Signals para armazenar os itens do carrinho, calcular o total, adicionar/remover itens e emitir um **output()** sempre que o total mudar.

```
import {  
  Component,  
  computed,  
  effect,  
  output,  
  signal,  
} from '@angular/core';  
  
interface ItemCarrinho {  
  id: number;  
  descricao: string;  
  quantidade: number;  
  preco: number;  
}  
  
@Component({  
  selector: 'app-contador-items-carrinho',  
  standalone: true,  
  template: `  
    <h2>Itens do Carrinho</h2>  
  
    <p>Total: {{ total() }}</p>  
  
    @for (item of listaItens(); track item.id) {  
      <div>  
        {{ item.descricao }}  
        Quantidade: {{ item.quantidade }}  
      </div>  
    }  
  `,  
})  
export class ItemCarrinhoComponent {  
  listaItens = signal<ItemCarrinho[]>([]);  
  total = computed(() => {  
    let total = 0;  
    for (const item of listaItens()) {  
      total += item.quantidade * item.preco;  
    }  
    return total;  
  });  
  totalOutput = output<number>();  
  effect(() => {  
    totalOutput.emit(total());  
  });  
}
```

```

        Preço: {{ item.preco }}

        <button type="button" (click)="removeItem(item.id)">
            Remover
        </button>
    </div>
}

<button
    type="button"
    (click)="addItem({
        id: 1,
        descricao: 'Produto',
        quantidade: 1,
        preco: 10
    })"
>
    Adicionar item
</button>
`
    },
    ))
export class ContadorItemsCarrinhoComponent {
    readonly listaItens = signal<ItemCarrinho[]>([]);

    readonly total = computed(() =>
        this.listaItens().reduce(
            (acc, item) => acc + item.quantidade * item.preco,
            0
        )
    );

    readonly totalChange = output<number>();

    constructor() {
        effect(() => {
            this.totalChange.emit(this.total());
        });
    }

    addItem(item: ItemCarrinho): void {
        this.listaItens.update((itens) => {
            const existente = itens.find((i) => i.id === item.id);

            if (existente) {
                return itens.map((i) =>
                    i.id === item.id
                        ? { ...i, quantidade: i.quantidade + item.quantidade }
                        : i
                );
            }

            return [...itens, item];
        });
    }

    removeItem(itemId: number): void {
        this.listaItens.update((itens) =>
            itens
                .map((item) =>
                    item.id === itemId
                        ? { ...item, quantidade: item.quantidade - 1 }
                        : item
                )
                .filter((item) => item.quantidade > 0)
        );
    }
}

```

3.2. Gerenciamento de Estado com NgRx (Feature To-do)

Implemente a estrutura de estado para uma lista de tarefas (To-do) utilizando os padrões recomendados do NgRx. A implementação deve incluir:

- **Actions:** Definir ações para: `loadTodos`, `loadTodosSuccess`, `loadTodosError` e `toggleTodoComplete`.
- **Reducer:** Implementar o estado inicial e a transição de estados usando `createReducer`, garantindo a tipagem forte do estado.
- **Selectors:** Criar seletores utilizando `createSelector`:
 - `selectAllTodos`: Retorna a lista completa.

- **selectPendingTodos:** Retorna apenas as tarefas não concluídas.
- **Effect:** Criar um **Effect** que gerencie o fluxo assíncrono: ao disparar **loadTodos**, deve realizar uma chamada HTTP (mockada) e despachar a ação de sucesso ou erro conforme o resultado.

Não é necessário implementar o back-end — use HttpClient com uma URL fictícia.

RESPOSTA

A feature To-do foi estruturada com actions, reducer, selectors, effect e service, mantendo o estado fortemente tipado.

MODEL

```
export interface Todo {
  readonly id: number;
  readonly title: string;
  readonly completed: boolean;
}
```

ACTIONS

```
import { createActionGroup, emptyProps, props } from '@ngrx/store';

export const TodosPageActions = createActionGroup({
  source: 'Todos Page',
  events: {
    'Load Todos': emptyProps(),
    'Toggle Todo Complete': props<{ id: number }>(),
  },
});

export const TodosApiActions = createActionGroup({
  source: 'Todos API',
  events: {
    'Load Todos Success': props<{ todos: ReadonlyArray<Todo> }>(),
    'Load Todos Error': props<{ error: string }>(),
  },
});
```

REDUCER

```
import { createFeature, createReducer, on } from '@ngrx/store';

export interface TodoState {
  readonly todos: ReadonlyArray<Todo>;
  readonly loading: boolean;
  readonly error: string | null;
}

export const initialTodosState: TodoState = {
  todos: [],
  loading: false,
  error: null,
};

export const todosReducer = createReducer(
  initialTodosState,
  on(TodosPageActions.loadTodos, (state): TodoState => ({
    ...state,
    loading: true,
    error: null,
  })),
  on(TodosApiActions.loadTodosSuccess, (state, { todos }): TodoState => ({
    ...state,
    todos,
    loading: false,
    error: null,
  })),
  on(TodosApiActions.loadTodosError, (state, { error }): TodoState => ({
    ...state,
    loading: false,
    error,
  })),
  on(TodosPageActions.toggleTodoComplete, (state, { id }): TodoState => ({
    ...state,
    todos: state.todos.map((todo) =>
```

```

        todo.id === id ? { ...todo, completed: !todo.completed } : todo
      ),
    )))
  );

export const todosFeature = createFeature({
  name: 'todos',
  reducer: todosReducer,
});

```

SELECTORS

```

import { createSelector } from '@ngrx/store';

export const selectAllTodos = todosFeature.selectAllTodos;

export const selectPendingTodos = createSelector(
  selectAllTodos,
  (todos) => todos.filter((todo) => !todo.completed)
);

export const selectError = todosFeature.selectError;
export const selectLoading = todosFeature.selectLoading;

```

SERVICE

```

import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

import { delay, Observable, of } from 'rxjs';
import { Todo } from '../models/todo.model';

@Injectable({
  providedIn: 'root',
})
export class TodosApiService {
  private readonly http = inject(HttpClient);
  private readonly apiUrl = 'https://api.exemplo.com/todos';

  getTodos(): Observable<ReadonlyArray<Todo>> {
    return this.http.get<ReadonlyArray<Todo>>(
      this.apiUrl
    );
  }

  getTodosMock(): Observable<ReadonlyArray<Todo>> {
    return of(todosMock).pipe(
      delay(800),
    );
  }
}

const todosMock: ReadonlyArray<Todo> = [
  {
    id: 1,
    title: 'Configurar o NgRx Store',
    completed: true,
  },
  {
    id: 2,
    title: 'Criar as actions da feature To-do',
    completed: true,
  },
  {
    id: 3,
    title: 'Implementar o reducer',
    completed: false,
  },
  {
    id: 4,
    title: 'Criar os selectors',
    completed: false,
  },
  {
    id: 5,
    title: 'Implementar o effect de carregamento',
    completed: false,
  },
  {

```

```

    id: 6,
    title: 'Integrar o componente com o Store',
    completed: false,
  },
  {
    id: 7,
    title: 'Validar o fluxo no Redux DevTools',
    completed: false,
  },
];

```

EFFECT

```

import { HttpResponse } from "@angular/common/http";
import { inject } from "@angular/core";
import { Actions, createEffect, ofType } from "@ngrx/effects";
import { TodosApiService } from "../services/todo.service";
import { TodosApiActions, TodosPageActions } from "../todo.actions";
import { catchError, exhaustMap, map, of } from "rxjs";

const getErrorMessage = (error: unknown): string => {
  if (error instanceof HttpResponse) {
    if (
      typeof error.error === 'object' &&
      error.error !== null &&
      'message' in error.error
    ) {
      return String(error.error.message);
    }

    return error.message ||
      `Erro HTTP ${error.status}`;
  }

  if (error instanceof Error) {
    return error.message;
  }

  return 'Não foi possível carregar as tarefas.';
}

export const loadTodosEffect = createEffect((
  actions$ = inject(Actions),
  service = inject(TodosApiService)
) => actions$.pipe(
  ofType(TodosPageActions.loadTodos),
  //Impedir várias requisições concorrentes
  exhaustMap(() => service.getTodos().pipe(
    map((todos) => TodosApiActions.loadTodosSuccess({todos})),
    catchError((error:unknown) => of(TodosApiActions.loadTodosError({
      error: getErrorMessage(error)
    })))
  )))
), {functional:true});

```

O **exhaustMap** foi usado no effect para evitar requisições concorrentes caso o usuário dispare o carregamento várias vezes antes da chamada anterior terminar.

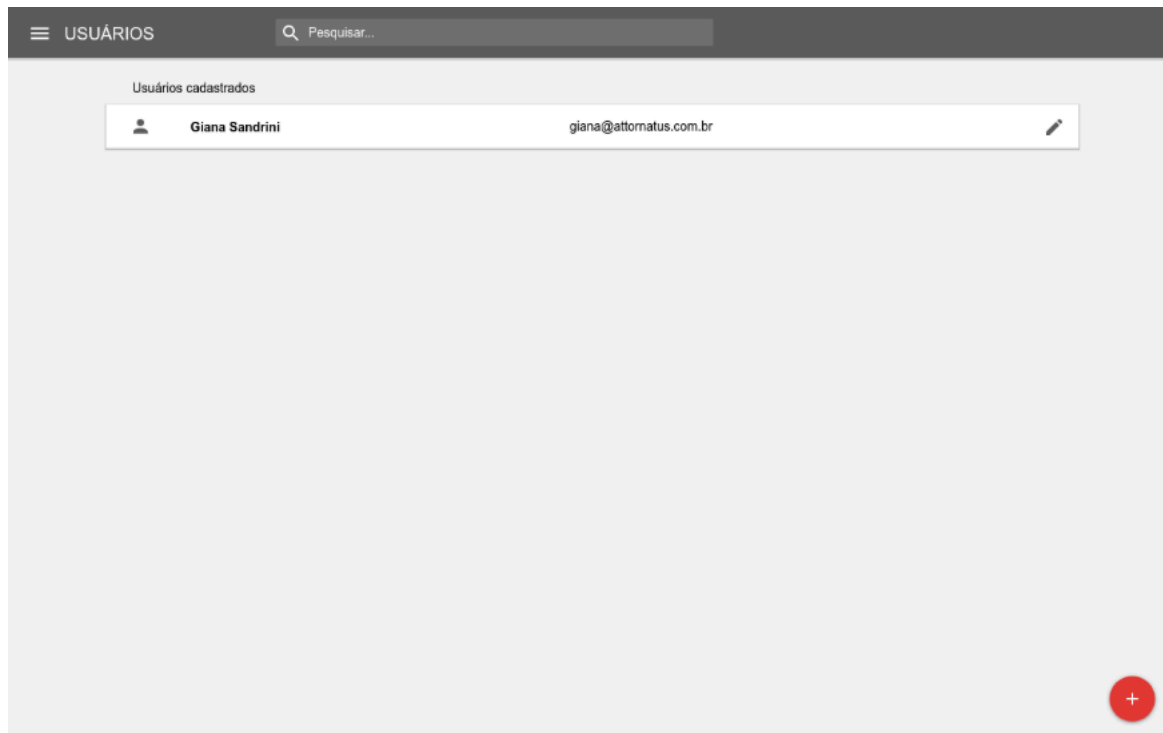
4. Desafio Prático — Aplicação Angular

Stack obrigatória: Angular 17+ · Angular Material · NgRx ou Signals · RxJS · Vitest ou Jest

4.1 O que construir

Reproduzir o protótipo de uma listagem de usuários. A listagem deverá ter as seguintes funcionalidades:

Tela de listagem de usuários:



Listagem de usuários:

- Cards com os campos: nome, e-mail e botão de editar
- Filtro por nome com debounce de 300ms
- Estado de loading durante o carregamento e mensagem de erro em caso de falha
- Os dados podem vir de uma API mockada (JSON Server, MSW ou array estático em serviço)

Modal de cadastro de novo usuário com abertura através do botão vermelho que aparece na listagem:

USUÁRIOS

Pesquisar...

Usuários cadastrados

Giana Sandrini giana@attomatus.com.br

Adicionar novo usuário

Usuário (e-mail) *

Nome completo *

CPF * Número do telefone * CELULAR

O usuário receberá uma senha provisória para acesso ao sistema por SMS.

SALVAR

Criação e edição de usuário (em modal):

- Formulário reativo com campos: e-mail (obrigatório), nome (obrigatório), cpf (obrigatório), telefone (obrigatório) e tipo de telefone.
- Validação com mensagens de erro por campo
- Botão de salvar desabilitado enquanto o formulário estiver inválido
- Quando for edição o formulário deve ser preenchido automaticamente

4.2 Requisitos técnicos

- Pelo menos 2 operadores RxJS além de map e tap em uso real (ex: switchMap, forkJoin, catchError, debounceTime)
- Componentes standalone
- Subscriptions gerenciadas sem memory leaks (takeUntilDestroyed, take, async pipe ou unsubscribe manual no ngOnDestroy)
- Cobertura de testes acima de 60% (Preferencialmente usando Vitest ou Jest)

4.3 Diferenciais (não obrigatórios)

- Nx Monorepo com separação em bibliotecas (ex: feature-users, data-access-users, ui)
 - Paginação na listagem
 - Validação de formato dos dados do formulário (e-mail, cpf e telefone)
 - Melhorias no projeto em relação a tela apresentada no protótipo
-

RESPOSTA AO DESAFIO PRÁTICO

Repositório público no GitHub: [Link do Projeto](#)